

대형 언어 모델(LLM)과 인간 코드의 품질 비교 분석

박성호¹, 박성진², 김장환³, 서채연⁴,김영철^{5*}

홍익대학교 소프트웨어융합과

(^{1,2}학부생, ³박사과정, ⁴ 초빙교수, ^{5*}교수)

andytdgh@gmail.com, sjw1111@naver.com,

{lentoconstante³, chaeyun⁴, bob^{5*}}@hongik.ac.kr

A Comparative Analysis of Code Quality Between Large Language Models (LLMs) and Human Developers

Sungho Park¹, Seong Jin Park²,

Janghwan Kim³, Chaeyun Seo⁴, R. Young Chul Kim^{5*}

^{1,2,3,4,5*}Dept. of Computer Science, Hongik University

요약

현재 ChatGPT, Gemini, Claude 등 대형 언어 모델(LLM)들이 소프트웨어 개발 현장에서 코드 생성을 활발히 수행하고 있다. 그러나 소프트웨어 공학(SE) 관점에서는 단순한 기능적 동작 여부를 넘어, 어떤 주체(LLM 또는 인간 개발자)가 생성한 코드가 실질적으로 품질과 신뢰성을 갖추고 있는가를 객관적으로 검증하는 것이 필수적이다. 이에 본 연구는 LLM과 인간 개발자 코드 간의 정량적 품질 비교 방안을 제안하며, AI 소프트웨어 품질 평가 지표로서 순환 복잡도(CC), 객체 결합도(CBO), 총 코드 라인 수(LOC)를 기준으로 삼았다. React/JavaScript 환경의 동일한 To-Do List 앱 구현 실험을 통해 유료 LLM 3종(ChatGPT, Gemini, Claude)과 소프트웨어 전공 학부생의 코드를 비교한 결과, LLM은 모듈화와 생산성 면에서 우위를 보였으나 오버엔지니어링 등 모델별 한계도 확인되었다. 본 연구는 이를 통해 SDLC 프로세스 내 CC·CBO 기반 정량적 품질 게이트 도입의 필요성을 실증하고, 개발자가 AI 산출물을 공학적으로 검증하는 아키텍트로서의 역할로 진화해야 함을 제안한다.

1. 서론

최근 소프트웨어 공학 분야는 대형 언어 모델(LLM)의 발전과 함께 전통적인 소프트웨어 개발 생명주기(SDLC)에 생성형 AI를 접목하는 패러다임 전환을 맞이하고 있다[1]. 특히 멀티모달 모델은 디자인 시안으로부터 즉각적인 소스 코드를 생성하여 구현 단계의 시간과 비용을 획기적으로 단축한다.

그러나 AI 산출물을 비판적 검토 없이 수용하는 것은 소프트웨어의 안정성을 저해하고 심각한 기술 부채(Technical Debt)를 초래할 위험이 크다[2]. AI 생성 코드는 기능적으로는 동작할지라도 구조적 정밀함이나 유지보수성 면에서 모델마다 큰 편차를 보이며, 오버엔지니어링이나 문맥적 일관성 결여 등의 결함이 빈번히 관찰되기 때문이다.

이에 본 연구는 무분별한 AI 코드 수용 방지를 위해 순환 복잡도(CC), 객체 결합도(CBO), 총 라인 수(LOC) 등 정량적 지표 기반의 ‘품질 게이트(Quality

Gate)’를 제안한다[3,4]. 핵심 목적은 개발자의 역량을 단순 ‘작성자(Coder)’에서 AI 산출물을 설계 관점에서 정제하는 ‘검증자 및 아키텍트(Validator & Architect)’로 재정의하는 데 있다.

2장에서는 LLM과 인간 개발자의 역량 대조 및 품질 측정 지표에 관한 관련 연구를 심층적으로 살펴보고, 3장에서는 실험 설계와 측정 결과를 분석하며, 4장에서는 결론 및 향후 연구 방향을 기술한다.

2. 에이전트 개발도구의 요구사항

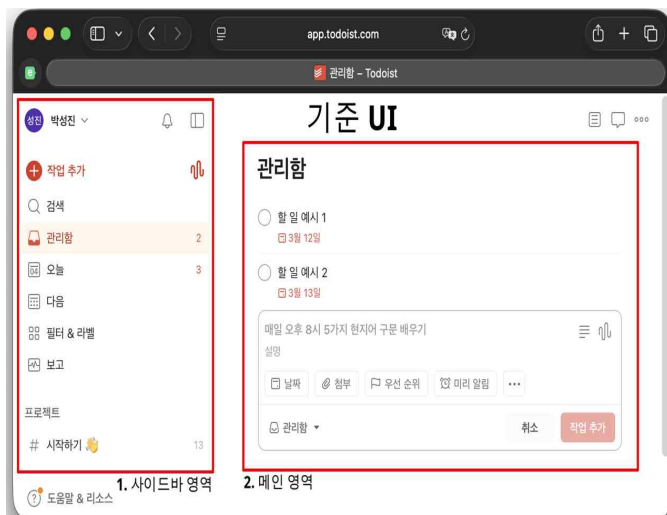
AI는 단일 알고리즘 최적화에서 인간을 상회하는 성능을 보이나, 복잡한 아키텍처 환경에서는 예외 처리 누락과 코드 중복으로 복잡도가 상승하는 양상을 보인다. 반면 학부생 개발자는 요구사항 이해력은 우수하나 아키텍처 준수나 모듈화 역량 면에서 AI와 대조적인 한계를 보인다[5]. 기존 문헌들은 주로 기능적 성공률에 집중해 왔으나, 이제는 유지

보수 비용과 직결되는 구조적 품질 지표에 대한 심층 분석이 필요한 시점이다. 이를 위한 핵심 도구로서 순환 복잡도(CC)와 객체 결합도(CBO)는 프로그램의 논리적 난해함과 모듈 간 의존성을 정량화한다 [4]. 복잡도가 높을수록 오류 확률이 증가하며, 결합도가 높을수록 수정의 파급 효과가 커진다. AI 도입으로 이러한 지표를 활용한 품질 통제의 중요성은 더욱 강조되고 있으며, 개발자는 AI 산출물을 품질 지표 기반으로 검증하는 'High-level' 통제 관점으로 진화해야 한다.

3. LLM과 인간 개발자 코드의 품질 비교

3.1 실험 환경 및 요구사항 명세

React와 JavaScript 환경에서 동일한 'To-Do List 앱' 구현 실험을 진행하였다. 실험 대상은 유료 LLM 3종(ChatGPT, Gemini, Claude)과 소프트웨어 전공자 2명으로 구성하였으며, 모든 피험자에게 동일한 조건의 프롬프트 및 요구사항 문서를 제공하여 실험의 공정성을 확보하였다. 코드 품질 측정은 자체 구축한 '코드 품질 분석 웹 애플리케이션'을 통해 수행하였으며, 순환 복잡도(CC), 객체 결합도(CBO), 총 라인 수(LOC) 등의 지표를 일괄 추출하였다. 기준 UI는 실무에서 널리 사용되는 태스크 관리 도구 Todoist를 참고하여 사이드바와 메인 영역으로 구성하였으며, 피험자에게는 ① 사이드바 내비게이션, ② 메인 영역 할 일 목록, ③ 원형 체크박스 완료 처리, ④ 하단 작업 입력 폼 재현 등 네 가지 요구사항을 명세하였다.



(그림 1) 실험 기준 UI 기준.

3.2 측정 결과 및 UI 분석

표 1은 추출된 정량적 지표 결과이다. 시각적 재현성 측면에서 Claude는 단 1회의 프롬프트와 6분의

소요 시간만으로 모든 컴포넌트를 기준 UI와 거의 완벽하게 일치시켜 95%의 최고 유사도를 기록하였다. 반면 학생 개발자는 평균 150분을 투입했음에도 사이드바와 하단 입력 폼의 복잡한 레이아웃을 구현하지 못해 유사도가 61%에 그쳤다. 이는 시각적 레이아웃 구성 단계에서 AI와 인간의 생산성 격차가 이미 극명하게 벌어졌음을 시사한다.

3.3 UI 재현성 세부 분석

시각적 재현성 측면에서 Claude는 단 1회의 프롬프트만으로 모든 컴포넌트를 기준 UI와 거의 완벽하게 일치시켜 95%의 최고 유사도를 기록했다. 특히 사이드바의 아이콘 배치와 입력 폼의 세부 버튼까지 정확히 구현했다. ChatGPT(90%)는 안정적이었으나 체크박스 형태에서 오차가 있었고, Gemini(80%)는 입력 폼 구성이 상이했다. 반면 학생 개발자는 사이드바와 하단 입력 폼 등 복잡한 레이아웃 구현을 생략한 단일 페이지를 볼 때 유사도가 61%로 가장 낮았다.

3.4 코드 품질 분석 및 기술 부채 고찰

품질 지표 분석 결과, 3종의 LLM은 공통으로 기능별 컴포넌트 분리와 높은 모듈화 성능을 보였다. ChatGPT(8개), Claude(8개), Gemini(6개) 모두 다수 파일로 역할을 분산하였으며, CC·CBO 수치 역시 인간 개발자 대비 낮은 수준을 유지해 구조적 유지보수성이 우수함을 확인하였다. 또한 소요 시간에서도 최대 33분 이내에 구현을 완료하여 생산성 측면의 압도적 우위를 입증하였다.

반면 인간 개발자는 LOC 221로 가장 짧은 코드를 작성하였으나, 구현을 단 3개의 파일에 집중시킴으로써 컴포넌트 단위의 기능 분리에 실패하였다. 그 결과 CC 7.15, CBO 3.9로 측정되어 전형적인 유지보수 불가형 '스파게티 코드'의 징후를 나타냈으며, 150분을 투입했음에도 UI 유사도는 61%에 그쳤다. 이는 인간 개발자가 바닥부터 코딩할 때 로직을 분산하기보다 하나의 파일에 밀집시키는 경향이 있음을 보여준다. 그러나 LLM 역시 모델별 한계가 존재한다. Claude는 UI 재현을 최고(95%)를 기록했음에도 LOC 868줄로 과도한 코드를 생성하였고, ChatGPT는 CC 1.56의 구조적 우수성에도 불구하고 요청 완결성이 낮았다. 결론적으로 LLM과 인간 개발자 모두 단일 지표에서의 우위가 전체 품질을 보장하지 않으며, 이는 구현 단계에서 CC·CBO 기반의 정량적 품질 통제 기준 도입이 필수적임을 실증적으로 뒷받침한다.

Claude UI		Gemini UI		Chat GPT UI		인간 개발자 UI	
							
UI 유사도	95%		80%		90%		61%
순환복잡도(CC)	4.79		3.0		1.56		7.15
Total LOC	868줄		526줄		263줄		221줄
소요시간	6분		33분		20분		150분

(그림 2) LLM 3종 및 인간 개발자의 UI 재현 결과 및 품질 지표 비교

4. 결론

본 연구는 상용 LLM 3종(ChatGPT, Gemini, Claude)과 소프트웨어 전공 학부생 간의 코드 생성 결과를 순환 복잡도(CC), 객체 결합도(CBO), 총 코드 라인 수(LOC) 등 정량적 지표를 기반으로 비교·분석하였다. 실험을 통해 인간 개발자의 로직 집중화로 인한 높은 복잡도 문제와 AI 에이전트의 압도적 효율성 및 모듈화 성능을 확인하였으며, Claude의 LOC 868이나 ChatGPT의 낮은 요청 완결성 등 모델별 한계 역시 명확히 드러났다. 이는 LLM 산출물이 기능적 완성도와 구조적 품질 간의 균형을 항상 보장하지 않음을 시사한다.

본 연구의 핵심 기여는 자체 개발한 '코드 품질 분석 애플리케이션'을 통해 AI 산출물의 오버엔지니어링 징후와 구조적 결함을 사전에 정량적으로 포착할 수 있음을 입증한 데 있다. 기존 연구들이 기능적 성공률 중심의 평가에 머물렀던 것과 달리, 본 연구는 유지보수 비용과 직결되는 구조적 품질 지표를 실험 기반으로 측정함으로써 소프트웨어 공학적 관점에서의 AI 코드 평가 체계를 구체화하였다.

결론적으로, 미래의 개발 조직은 SDLC 프로세스 내에 CC·CBO 기반의 정량적 '품질 게이트'를 구축하고, 개발자는 AI 산출물을 공학적으로 검증·최적화하는 '아키텍트'로서의 역할을 수행해야 한다. 향후 연구에서는 실제 CI/CD 환경에서 자동 리팩토링을 제안하는 지능형 품질 통제 시스템의 구축 효과를 분석하고, 다양한 도메인으로 실험 범위를 확장하여 본 연구의 일반화 가능성을 검증할 계획이다.

Acknowledgment

본 연구는 2026 홍익대학교 소프트웨어융합학과 중

합설계 프로젝트 결과물이자, 메타버스 융합SW아카데미(과제번호: 202301830004) 6기 프로젝트 결과물이다.

참고문헌

- [1] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J. M. Zhang, "Large language models for software engineering: Survey and open problems," 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), IEEE, pp.31-53, 2023.
- [2] A. Al Mujahid, Mia Mohammad Imran, "TODO: Fix the mess Gemini created: Towards understanding GenAI-induced self-admitted technical debt," arXiv preprint arXiv:2601.07786, 2026.
- [3] T. J. McCabe, "A complexity measure," IEEE Transactions on Software Engineering, vol.4, pp.308-320, 1976.
- [4] S.Chidamber, C.Kemerer, "A metrics suite for object oriented design,"IEEE Transactions on Software Engineering, vol.20, no.6, pp.476-493, 1994.
- [5] M. T. Jamil, S. Abid, S. Shamail, "Can LLMs Generate Higher Quality Code Than Humans? An Empirical Study," 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), IEEE, pp.478-489, 2025.